~ Complete Step by Step ~

HTML Notes

From Zero to Hero · All Topics Covered

Index

1 Introduction to HTML
2 Basic Structure of a Page
3 Headings, Paragraphs & Comments
4 Text Formatting Tags
5 Quotes, Code & Special Characters
6 Attributes & Global Attributes
7 Links (Anchor Tag)
8 Images
9 Audio, Video, iframe & Favicon
10 Lists
11 Tables
12 Block, Inline, div & span
13 Semantic HTML5 Layout
14 Forms – Part 1 (input)
15 Forms – Part 2 (other elements)
16 Head, Meta, SEO & Responsive
17 CSS & JavaScript with HTML
18 HTML5 APIs & Graphics
19 Accessibility & Best Practices
20 Full Practice Page
21 Interview Q&A
22 Cheat Sheet

How to use these notes

- → Read the yellow highlights – they are the most important points.
- → **Red** = tag names, **blue** = attributes, **green** = values / output.
- → Type every example in your editor and run it in the browser. Practice makes perfect!

Notes by asadokz18 ✿

1. Introduction to HTML

What is HTML?

- HTML = Hyper Text Markup Language.
- **Hypertext** → text that contains links to other pages.
- **Markup** → tags that tell the browser how to show content.
- It is the skeleton / structure of every web page. It is not a programming language.
- Created by Tim Berners-Lee in 1991. Now maintained by W3C & WHATWG.

Web = 3 layers

HTML → Structure (bones)
CSS → Style (clothes)
JavaScript → Behaviour (brain)

Key words

Tag -
Element - start tag + content + end tag
Attribute - extra info in start tag

Anatomy of an Element

```
<p class="intro">Hello World</p>
```

 opening tag • class="intro" attribute • Hello World = content • closing tag.

Empty (void) elements have no closing tag:
 <hr> <img> <input> <meta> <link>.

History & Versions

Year	Version	What was new
1991	HTML 1.0	First tags by Tim Berners-Lee
1995	HTML 2.0	Forms, tables basics
1997	HTML 3.2 / 4.0	CSS support, scripting, frames
2000	XHTML 1.0	Strict XML-style rules
2014	HTML5	Semantic tags, audio, video, canvas, APIs

How the Web Works

1. You type a URL in the browser (client).
2. Browser asks the **server** for the page (HTTP/HTTPS request).
3. Server sends back HTML (+ CSS, JS, images).
4. Browser reads and renders HTML as a page.

What do you need?

- Text editor: VS Code, Notepad++, Sublime
- Browser: Chrome, Firefox, Edge
- Save files as .html or .htm
- Open file → double-click → browser

Remember

HTML is **not case-sensitive** (<P> =) but always write tags in lowercase.

2. Basic Structure of an HTML Page

Skeleton – index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

Every part explained

Part	Meaning
`<!DOCTYPE html>`	Tells the browser this is an **HTML5** document. Must be the very first line. (Not a tag – a declaration.)
`<html>`	Root element. Everything lives inside. `lang="en"` sets page language.
`<head>`	Hidden info **about** the page: title, meta, CSS links, scripts.
`<meta charset>`	Character set. **UTF-8** supports all languages & emojis.
`<meta viewport>`	Makes the page **mobile friendly** (responsive).
`<title>`	Text shown in the **browser tab** and search results.
`<body>`	All visible content: text, images, links, tables...

Tree view (DOM)

```
html
├── head
│   ├── meta
│   └── title
└── body
    ├── h1
    └── p
```

Steps to create your first page

1. Open an editor and create a new file.
2. Type the skeleton above.
3. Save as **index.html**.
4. Double-click it → opens in the browser. 🎉

Common mistakes

- ➜ Forgetting the closing tag. • Wrong nesting: `<b><i>text</b></i>` ✘ → `<b><i>text</i></b>` ✔
- ➜ Forgetting quotes in attribute values. • Saving file as **.txt** instead of .html.

3. Headings, Paragraphs & Comments

Headings <h1> to <h6>

- 6 levels. <h1> is **biggest & most important**, <h6> is smallest.
- Use only one <h1> per page (main title) – good for SEO.
- Don't pick headings for size – use them for **structure**.

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

Heading 1
Heading 2
Heading 3
Heading 4
Heading 5
Heading 6

Paragraph <p>

- Defines a paragraph; browser adds space before & after it.
- Extra **spaces and new lines in code are ignored** – browser shows one space only.

```
<p>This is a paragraph.</p>
<p>This is another   paragraph with    extra spaces.</p>
```

Line Break
 & Horizontal Line <hr>

 → breaks the line (no new paragraph). Empty tag.

```
Roses are red,<br>Violets are blue
```

<hr> → draws a **horizontal rule** (line) to separate topics.

```
<p>Topic 1</p>
<hr>
<p>Topic 2</p>
```

Comments

Comments are **not shown** in the browser. They are notes for developers.

```
<!-- This is a single-line comment -->
<!--
   This is a
   multi-line comment
-->
```

Shortcut in VS Code: **Ctrl + /**

Tip

Don't use many
 tags to make gaps – use **CSS margin** instead. Use <p> for real paragraphs.

4. Text Formatting Tags

Formatting tags change how text *looks* or what it *means*. Two types: physical (look only) and logical / semantic (give meaning).

Tag	Output	Use / Meaning
<b>	**Bold text**	Bold, no extra importance
	Strong text	Important text (bold) – semantic
<i>	*Italic text*	Italic – technical terms, thoughts
	Emphasised	Stress / emphasis (italic) – semantic
<u>	Underlined	Underline (misspelled / names)
<mark>	Highlighted	Highlights / marks text
<small>	Smaller text	Fine print, copyright
<del>	~~Deleted~~	Removed text (strike-through)
<s>	~~No longer valid~~	Strike – no longer accurate
<ins>	Inserted	Added text (underline)
<sub>	H_2O	Subscript – below the line
<sup>	x^2	Superscript – above the line
<big>	–	Removed in HTML5 – don't use!

Example

```
<p>This is <b>bold</b> and <i>italic</i> and <mark>highlighted</mark>.</p>
<p>Water is H<sub>2</sub>O and area = a<sup>2</sup></p>
<p>Price: <del>$50</del> <ins>$30</ins></p>
```

Important difference

<b> / <i> only change the **look**. / tell browsers & **screen readers** that the text is important – better for accessibility and SEO. Prefer and .

Other text-level tags

- → <abbr title="HyperText"> – abbreviation, shows tooltip on hover
- → <address> – contact info (shown in italic)
- → <bdo dir="rtl"> – reverse text direction
- → <time datetime="2026-10-14"> – date / time
- → <wbr> – possible word-break point
- → <dfn> – defining term

```
<p><abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>
<address>Written by asadokz18<br>Pakistan</address>
```

5. Quotes, Code & Special Characters

Quotation tags

Tag	Use
<blockquote>	Long quotation. Shown as an **indented block**. Attribute cite="url" gives the source.
<q>	Short, **inline** quotation. Browser adds quotation marks " ".
<cite>	Title of a work (book, movie, song) – shown in italic.

```
<blockquote cite="https://example.com">Success is the sum of small efforts.</blockquote>
<p>He said, <q>Learn HTML first.</q></p>
<p><cite>The Pragmatic Programmer</cite> is a great book.</p>
```

Computer-code tags

Tag	Meaning	Output (monospace)
<code>	A piece of computer code	`x = 5;`
<kbd>	Keyboard input	`Ctrl + C`
<samp>	Sample output from a program	`Error 404`
<var>	A variable	*y* = 2*x*
<pre>	Preformatted block (keeps spacing)	`line 1`

<pre> keeps spaces & line breaks exactly as typed:

```
<pre>  *
  ***
 *****</pre>
```

Special characters (HTML Entities)

- Some characters are reserved: <, >, &. If you type «, browser thinks a tag starts!
- Use an **entity**: starts with **&** and ends with **;** – name or number.

Entity	Shows	Meaning	Entity	Shows	Meaning
<	<	less than	©	©	copyright
>	>	greater than	®	®	registered
&	&	ampersand	™	™	trademark
"	"	double quote	€	€	euro
	␣	non-breaking space	♥	♥	heart

```
<p>5 &lt; 10 and 10 &gt; 5</p>
<p>Tom &amp; Jerry &copy; 2026</p>
<p>Hello   World</p>
```

Emoji

Because of UTF-8 you can use emojis directly 😀 or with a code like **😀**.

6. Attributes & Global Attributes

- Attributes give extra information about an element.
- Always written in the **start tag** as *name="value"*. Use double quotes.
- Many attributes can be used together, separated by a space.

```
<img src="photo.jpg" alt="My photo" width="200" height="150">
```

Global attributes (work on every element)

Attribute	Purpose
id	**Unique** name of one element (used by CSS, JS and anchor links). Only one per page.
class	Group name; **many elements** can share it; one element can have many classes.
style	Inline CSS: *style="color:red;"*
title	Tooltip text shown when mouse hovers.
lang	Language of content: *lang="en"*, *lang="ur"*.
dir	Text direction: ltr / rtl (Urdu, Arabic use rtl).
hidden	Hides the element from the page.
contenteditable	Makes the content editable by the user.
draggable	Element can be dragged (true / false).
tabindex	Order of focus when pressing the Tab key.
spellcheck	Turn spell checking on / off.
*data-**	Custom data: *data-user="asad"* (read by JavaScript).

Examples

```
<h1 id="main-title">Welcome</h1>
<p class="note big" title="Hover me!" style="color:blue;">Styled paragraph</p>
<p contenteditable="true">You can edit this text.</p>
<p hidden>You cannot see me.</p>
<div data-user="asadokz18" data-level="5">Custom data</div>
```

id vs class

id → unique, written ***#id*** in CSS.

class → reusable, written ***.class*** in CSS.

Boolean attributes

Only the name is needed: *disabled*, *checked*, *required*, *readonly*, *autofocus*, *hidden*, *controls*.

7. Links – the Anchor Tag <a>

- Links connect web pages. Created with the <a> (anchor) tag and the *href* attribute (Hypertext REFerence).
- The text between the tags is the **clickable link text**. Default: blue underlined unvisited, purple visited, red active.

```
<a href="https://www.google.com">Visit Google</a>
```

Types of links

Type	Example
Absolute	*href="https://example.com/page.html"* – full URL, other websites
Relative	*href="about.html"* or *href="pages/contact.html"* or *href="../index.html"*
Email	*href="mailto:abc@gmail.com"* – opens mail app
Phone	*href="tel:+923001234567"* – calls on mobile
Bookmark	*href="#section2"* – jumps to element with id="section2"
File	*href="notes.pdf" download* – downloads the file

Important attributes of <a>

Attribute	Value & meaning
href	Destination address (URL).
target	**_self** – same tab (default)
	_blank – new tab / window
	_parent – parent frame | **_top** – full window
	framename – inside the named iframe
rel	Relationship. Use ***noopener noreferrer*** with _blank for security; ***nofollow*** for SEO.
download	Downloads the file instead of opening it.
title	Tooltip on hover.

Examples

```
<a href="https://wikipedia.org" target="_blank" rel="noopener noreferrer">Open in new tab</a>

<!-- Link on an image -->
<a href="home.html"><img src="logo.png" alt="Home"></a>

<!-- Bookmark / jump link -->
<a href="#chapter2">Go to Chapter 2</a>
...
<h2 id="chapter2">Chapter 2</h2>

<a href="mailto:me@gmail.com?subject=Hello">Email me</a>
<a href="tel:+923001234567">Call me</a>
```

8. Images

- Use the <img> tag – it is an **empty tag** (no closing tag).
- Two required attributes: *src* (source path) and *alt* (alternative text).

```
<img src="images/cat.jpg" alt="A sleeping cat" width="300" height="200">
```

Attribute	Meaning
src	Path or URL of the image file.
alt	Text shown if image fails to load; read by **screen readers**; used by search engines.
width height	Size in pixels. Set both to avoid page jumping while loading.
loading="lazy"	Loads image only when it comes near the screen → faster page.
srcset sizes	Different image files for different screen sizes.
usemap	Connects the image to an image map.

Image formats

JPG / JPEG
Photos, small size

PNG
Transparent background

GIF
Simple animation

SVG
Vector, never blurry

WebP
Modern, very small

ICO
Favicon icons

Figure with caption

```
<figure>
  <img src="mountain.jpg" alt="Snowy mountain">
  <figcaption>Fig 1: Mountains in winter</figcaption>
</figure>
```

Responsive images – <picture> & srcset

```
<picture>
  <source media="(min-width: 800px)" srcset="large.jpg">
  <source media="(min-width: 400px)" srcset="medium.jpg">
  <img src="small.jpg" alt="Responsive image">
</picture>
```

Image map (clickable areas)

```
<img src="planets.jpg" alt="Planets" usemap="#planetmap">
<map name="planetmap">
  <area shape="rect" coords="34,44,270,350" href="sun.html" alt="Sun">
  <area shape="circle" coords="337,300,44" href="venus.html" alt="Venus">
</map>
```

9. Audio, Video, iframe & Favicon

Video <video>

```
<video width="400" controls autoplay muted loop poster="thumb.jpg">
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

Audio <audio>

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support audio.
</audio>
```

Attribute	Meaning
controls	Shows play / pause / volume buttons.
autoplay	Starts automatically (browsers need **muted** with it).
loop	Repeats again and again.
muted	Starts with sound off.
poster	Preview image before video plays (video only).
preload	none / metadata / auto – how much to load in advance.
width height	Player size (video).

Formats

Video: MP4, WebM, Ogg
Audio: MP3, WAV, Ogg

Subtitles

<track kind="subtitles" src="en.vtt" srclang="en" label="English"> inside <video>.

iframe – a page inside a page

- Embeds another web page, map or YouTube video.
- Attributes: src width height title name loading="lazy" allowfullscreen.

```
<iframe src="https://www.youtube.com/embed/VIDEO_ID" width="420" height="260" title="Video"
allowfullscreen></iframe>
<iframe src="page.html" name="myframe"></iframe>
<a href="other.html" target="myframe">Open inside frame</a>
```

Favicon (tab icon)

Small icon shown in the browser tab. Put this inside <head>:

```
<link rel="icon" type="image/png" href="favicon.png">
```

Note

Some websites block iframes (X-Frame-Options). Don't embed untrusted pages – use sandbox for safety.

10. Lists

Three types: unordered (bullets), ordered (numbers) and description lists.

1) Unordered list <ul> +

```
<ul>
  <li>Apple</li>
  <li>Mango</li>
  <li>Banana</li>
</ul>
```

- Apple
- Mango
- Banana

CSS list-style-type: **disc** ● (default) · **circle** ○ · **square** ■ · **none**.

2) Ordered list

```
<ol type="A" start="3" reversed>
  <li>First</li>
  <li>Second</li>
</ol>
```

Attribute	Meaning
type	1 (numbers, default) · A · a · I (Roman) · i
start	Number to start from: start="5" → 5, 6, 7...
reversed	Counts backward (3, 2, 1).
value	On - sets the number of that item.

3) Description list <dl> <dt> <dd>

→ <dl> list · <dt> = term / name · <dd> = description.

```
<dl>
  <dt>HTML</dt>
  <dd>Language for web page structure</dd>
  <dt>CSS</dt>
  <dd>Language for styling</dd>
</dl>
```

Nested list (list inside list)

```
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Orange</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```

Remember

Nested list goes **inside an** , not directly in <ul>.

Menus / navigation bars are made with <ul> + CSS!

11. Tables

Tables show data in **rows and columns**. Never use tables for page layout!

```
<table border="1">
  <caption>Student Marks</caption>
  <thead>
    <tr><th>Name</th><th>Subject</th><th>Marks</th></tr>
  </thead>
  <tbody>
    <tr><td>Asad</td><td>HTML</td><td>95</td></tr>
    <tr><td>Ali</td><td>CSS</td><td>88</td></tr>
  </tbody>
  <tfoot>
    <tr><td colspan="2">Total</td><td>183</td></tr>
  </tfoot>
</table>
```

Tag	Meaning
`<table>`	Whole table
`<tr>`	Table Row
`<th>`	Table Heading cell – bold & centred
`<td>`	Table Data cell
`<caption>`	Title of the table (placed first inside `<table>`)
`<thead>` `<tbody>` `<tfoot>`	Header group, body group, footer group
`<colgroup>` `<col>`	Style / group whole columns

Merging cells

colspan

Cell spans **columns** (left → right).
`colspan="2"`

rowspan

Cell spans **rows** (top → down).
`rowspan="3"`

```
<tr><th colspan="2">Full Name</th></tr>
<tr><td rowspan="2">Group A</td><td>Asad</td></tr>
<tr><td>Ali</td></tr>
```

Styling tables (CSS)

```
table { border-collapse: collapse; width: 100%; }
th, td { border: 1px solid #333; padding: 8px; text-align: left; }
th { background: #4a90e2; color: white; }
tr:nth-child(even) { background: #f2f2f2; }
```

- Other CSS: ***padding***, ***text-align***, ***border-spacing***, ***:hover*** for row highlight.
- Old attributes (cellpadding, cellspacing, align, bgcolor) are **not used in HTML5**.

12. Block, Inline, &

Block-level elements

- Always start on a **new line**.
- Take the **full width** available.
- Can have width, height, margin, padding.
- ***div, p, h1–h6, ul, ol, li, table, form, header, footer, section, article, nav, pre, hr***

Inline elements

- Stay **in the same line**.
- Take only as much width as content.
- Width / height are ignored.
- ***span, a, b, i, strong, em, img, input, button, label, code, small, mark, br***

inline-block

Mixed: stays in line **like inline** but allows width/height **like block** (CSS *display: inline-block*).

– Division (block container)

- A generic box with **no meaning**. Groups elements for layout and styling.

```
<div class="card" style="background:lightblue; padding:10px;">
  <h2>Title</h2>
  <p>Text inside the box</p>
</div>
```

– inline container

- Used to style a **small part of text** inside a line.

```
<p>My favourite colour is <span style="color:red; font-weight:bold;">red</span>.</p>
```

*Output: My favourite colour is **red**.*

Block level	Inline level
Starts new line	Same line
Groups big sections / layout	Styles small text pieces
Can contain block + inline	Should contain only inline

Page layout with div (old way)

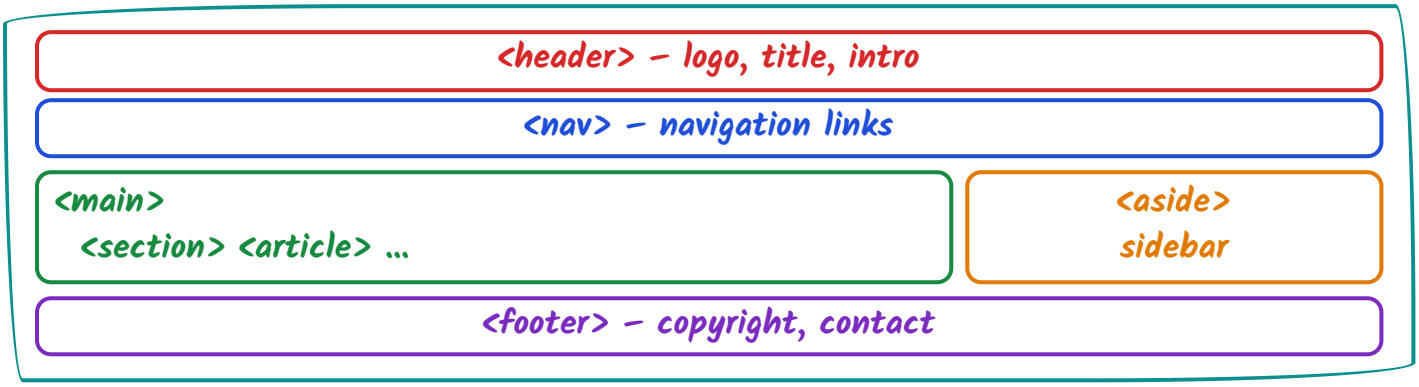
```
<div id="header">Header</div>
<div id="menu">Menu</div>
<div id="content">Main content</div>
<div id="footer">Footer</div>
```

Better way

Use **semantic tags** (<header>, <nav>, <main>, <footer>) instead of lots of divs – next chapter!

13. Semantic HTML5 Layout

Semantic = tag names that describe their meaning (unlike and). Helps browsers, search engines and screen readers.



Tag	Purpose
<header>	Top of page / section: logo, heading, intro.
<nav>	Main **navigation** links (menu).
<main>	Main unique content. Only **one** per page.
<section>	A themed group of content with a heading.
<article>	Self-contained content: blog post, news, comment.
<aside>	Side content: ads, related links, sidebar.
<footer>	Bottom: copyright, links, contact info.
<figure> <figcaption>	Image / diagram with caption.
<details> <summary>	Collapsible content (click to show / hide).
<dialog> <time>	Pop-up dialog box · date / time value.

Example page

```
<body>
  <header><h1>My Blog</h1></header>
  <nav><a href="/">Home</a> <a href="/about">About</a></nav>
  <main>
    <article>
      <h2>HTML is fun</h2>
      <p>Posted on <time datetime="2026-10-14">14 Oct 2026</time></p>
    </article>
    <aside>Related posts...</aside>
  </main>
  <footer>&copy; 2026 asadokz18</footer>
</body>
```

details & summary

```
<details open><summary>Click to read more</summary><p>Hidden text appears here.</p></details>
```

14. Forms – Part 1 (input)

➜ Forms collect **user data** (login, signup, search, contact) and send it to a server.

```
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="username" placeholder="Enter name" required>
  <input type="submit" value="Send">
</form>
```

Form attribute	Meaning
action	URL / file where the data is sent.
method	GET – data in URL (search, not secret). POST – data hidden (passwords, big data).
target enctype	Where to show response · multipart/form-data for file upload.
autocomplete novalidate	Browser suggestions on/off · skip validation.

<input> types

type	Use
text	Single-line text
password	Hidden dots ●●●
email	Valid email check
number	Numbers (min,max,step)
tel	Phone number
url	Web address
search	Search box
date	Date picker
time	Time picker

type	Use
datetime-local	Date + time
month / week	Month / week
color	Colour picker
range	Slider
file	Upload file
checkbox	Many choices ☑
radio	One choice ◉
hidden	Invisible data
submit/reset/button	Buttons

Checkbox & Radio (radios of one group share the same name → only one choice)

```
<input type="checkbox" id="c1" name="hobby" value="coding" checked>
<label for="c1">Coding</label>

<input type="radio" id="m" name="gender" value="male">
<label for="m">Male</label>
<input type="radio" id="f" name="gender" value="female">
<label for="f">Female</label>
```

Important input attributes

name key sent to server (needed to submit!) · **value** default · **placeholder** hint · **required** · **readonly** (sent) · **disabled** (not sent) · **autofocus** · **maxlength minlength** · **min max step** · **pattern** regex · **size**

15. Forms – Part 2 (other elements)

<label>

Names an input. Clicking the label focuses the input. Link them: **for="id"** = input's **id**. Or wrap the input inside the label.

<textarea> – multi-line text

```
<textarea name="msg" rows="4" cols="30" placeholder="Your message" maxlength="200"></textarea>
```

<select> – dropdown list

```
<select name="city" id="city">
  <optgroup label="Punjab">
    <option value="lhr">Lahore</option>
    <option value="mul" selected>Multan</option>
  </optgroup>
  <option value="khi">Karachi</option>
</select>
```

- → **<option>** – each choice
- → **selected** – default choice
- → **<optgroup>** – groups options
- → **multiple** – choose many
- → **size** – rows visible

<datalist> – suggestions while typing

```
<input list="browsers" name="browser">
<datalist id="browsers"><option value="Chrome"><option value="Firefox"><option value="Edge"></datalist>
```

<button>

```
<button type="submit">Submit</button>
<button type="reset">Reset</button>
<button type="button" onclick="alert('Hi!')">Click me</button>
```

<fieldset> & <legend> – group fields

```
<fieldset>
  <legend>Personal Info</legend>
  <label>Name: <input type="text"></label>
</fieldset>
```

More form elements

- → **<output>** – shows calculation result
- → **<progress value="70" max="100">** – progress bar
- → **<meter value="0.6">** – gauge / scale
- → **form** attribute – input outside form

Validation (built-in)

HTML5 validation

required · **type="email"** · **pattern="[0-9]{11}"** · **min / max** · **minlength / maxlength**. Browser stops the submit and shows a message if wrong. Still validate on the server too!

16. Head, Meta, SEO & Responsive

The <head> holds info **about** the page. Not shown on screen. Allowed tags: <title> <meta> <link> <style> <script> <base>.

<meta> tags

```
<meta charset="UTF-8">
<meta name="description" content="Free HTML notes for beginners">
<meta name="keywords" content="HTML, CSS, notes">
<meta name="author" content="asadokz18">
<meta name="robots" content="index, follow">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="refresh" content="30">
```

Meta	Purpose
charset	Character encoding – always UTF-8.
description	Short summary shown in search results (≈150 characters).
viewport	Mobile screen width & zoom – needed for responsive design.
robots	Tell search engines to index or not (index / noindex, follow / nofollow).
http-equiv="refresh"	Auto reload every 30 s or redirect after some seconds.

<link>, <base>

```
<link rel="stylesheet" href="style.css">
<link rel="icon" href="favicon.png">
<link rel="preconnect" href="https://fonts.googleapis.com">
<base href="https://mysite.com/" target="_blank">
```

SEO basics (Search Engine Optimization)

- Unique, meaningful <title> (50–60 characters) and meta description.
- One <h1>; headings in order h1 → h2 → h3.
- Use semantic tags and descriptive **alt** text for images.
- Clean link text, mobile-friendly, fast page. Social cards: <meta property="og:title">.

Responsive web design

- Page looks good on mobile, tablet and desktop.
- Steps: ① add viewport meta ② flexible widths (**%**, **max-width:100%**) ③ CSS media queries ④ responsive images.

```
img { max-width: 100%; height: auto; }
@media (max-width: 600px) {
  body { font-size: 14px; }
  .menu { display: none; }
}
```

17. CSS & JavaScript with HTML

Adding CSS – 3 ways

Way	How	Use
Inline	**style** attribute on the element	One element only (highest priority)
Internal	**<style>** tag inside **<head>**	Single page
External	**<link rel="stylesheet" href="style.css">**	Best – many pages share one file

```
<!-- 1. Inline -->
<p style="color:red; font-size:20px;">Red text</p>
<!-- 2. Internal -->
<head><style> p { color: blue; } </style></head>
<!-- 3. External -->
<head><link rel="stylesheet" href="style.css"></head>
```

CSS syntax & selectors

```
selector { property: value; }

p { color: green; }              /* tag selector   */
.note { background: yellow; }  /* class selector */
#title { font-size: 40px; }    /* id selector    */
```

Cascade order (low → high): **browser default** → **external** → **internal** → **inline**.

Adding JavaScript

➜ JavaScript makes the page **interactive** – clicks, popups, validation, animation.

```
<!-- Internal -->
<script>
  document.getElementById("demo").innerHTML = "Hello JS!";
</script>
<!-- External (before closing body, or use defer) -->
<script src="app.js" defer></script>
<!-- Event inside HTML -->
<button onclick="alert('Clicked!')">Click</button>
<p id="demo"></p>
```

Attribute	Meaning
defer	Download while HTML loads, run **after** page is parsed (keeps order).
async	Download & run as soon as ready (order not guaranteed).

<noscript>Please enable JavaScript**</noscript>** → shown if JS is off. Common events: **onclick, onchange, onmouseover, onkeyup, onload, onsubmit**.

18. HTML5 APIs & Graphics

Canvas – draw with JavaScript

```
<canvas id="c" width="200" height="100" style="border:1px solid"></canvas>
<script>
  const ctx = document.getElementById("c").getContext("2d");
  ctx.fillStyle = "orange";
  ctx.fillRect(20, 20, 100, 50);   // x, y, width, height
</script>
```

→ Pixel based (like a painting) – games, charts, animations. Methods: fillRect, strokeRect, arc, lineTo, fillText, drawImage.

SVG – Scalable Vector Graphics

```
<svg width="200" height="100">
  <circle cx="50" cy="50" r="40" fill="orange" stroke="navy" stroke-width="4" />
  <rect x="110" y="20" width="70" height="60" fill="teal" />
</svg>
```

→ XML-based, shapes stay sharp when zoomed. Shapes: circle, rect, line, ellipse, polygon, path, text.

Canvas	SVG
Pixel based, drawn by JS	Vector based, shapes written in HTML
Fast for games / many objects	Sharp at any size, easy CSS styling

Web Storage API

→ Store data in the browser. **localStorage** – stays forever. **sessionStorage** – until tab closes. (~5 MB, better than cookies)

```
localStorage.setItem("user", "asadokz18");
"K">let u = localStorage.getItem("user");
localStorage.removeItem("user");
```

Geolocation API

```
navigator.geolocation.getCurrentPosition(p => {
  console.log(p.coords.latitude, p.coords.longitude);
});
```

Drag and Drop

```
<img id="pic" src="a.png" draggable="true" ondragstart="drag(event)">
<div ondrop="drop(event)" ondragover="event.preventDefault()"></div>
```

Other APIs

Web Workers · Fetch · Notifications · History · Web Sockets · Canvas & WebGL · Clipboard

Privacy

Geolocation always asks the user for permission first.

19. Accessibility & Best Practices

Accessibility (a11y) = making pages usable by everyone, including blind users (screen readers), keyboard-only users and people with other disabilities.

Accessibility checklist

- Meaningful **alt** text on images (empty *alt=""* for decoration).
- Use semantic tags: *<nav>* *<main>* *<button>* (not a div acting as a button).
- Always connect *<label>* to inputs. Use proper heading order.
- Set *lang* on *<html>*. Good colour contrast. Page works with **keyboard only** (Tab, Enter).
- Don't rely on colour alone. Add captions / subtitles for video & audio.

ARIA attributes (Accessible Rich Internet Applications)

Attribute	Meaning
role	Defines the role: *role="navigation"*, *role="alert"*, *role="button"*
aria-label	Invisible text label for screen readers.
aria-labelledby	Use another element's text as the label.
aria-hidden="true"	Hide decorative element from screen readers.
aria-expanded	Shows if a menu / section is open (true / false).
aria-live	Announce dynamic updates (polite / assertive).

```
<button aria-label="Close menu">×</button>
<nav aria-label="Main menu">...</nav>
<img src="deco.png" alt="" role="presentation">
```

First rule of ARIA: **use a native HTML element first**; add ARIA only when no tag fits.

Best practices

- Always start with *<!DOCTYPE html>*.
- Lowercase tags & attribute names.
- Close all tags; nest correctly.
- Quote attribute values.
- Indent code (2 or 4 spaces).
- Use meaningful file / id / class names.
- No inline styles – use external CSS.
- Specify **alt**, width & height on images.
- Keep files in folders (css/, js/, images/).
- Use *loading="lazy"* & compress images.
- Validate with **validator.w3.org**.
- Comment your code, test on many browsers.

Security

Use **https** · add *rel="noopener noreferrer"* on _blank links · never trust user input · validate on the server.

20. Full Practice Page

Let's use everything we learned to build a small personal website page!

index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Personal page of asadokz18">
  <title>asadokz18 | My Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <img src="logo.svg" alt="asadokz18 logo" width="50">
    <h1>asadokz18</h1>
    <nav><a href="#about">About</a> | <a href="#skills">Skills</a> | <a href="#contact">Contact</a></nav>
  </header>
  <main>
    <section id="about">
      <h2>About Me</h2>
      <p>I am learning <strong>HTML</strong> &amp;amp; <em>CSS</em>.</p>
    </section>
    <section id="skills">
      <h2>My Skills</h2>
      <ul><li>HTML</li><li>CSS</li><li>JavaScript</li></ul>
      <table border="1">
        <tr><th>Skill</th><th>Level</th></tr>
        <tr><td>HTML</td><td>Good</td></tr>
      </table>
    </section>
    <section id="contact">
      <h2>Contact</h2>
      <form action="/send" method="post">
        <label for="e">Email:</label>
        <input type="email" id="e" name="email" required>
        <textarea name="msg" placeholder="Message"></textarea>
        <button type="submit">Send</button>
      </form>
    </section>
  </main>
  <footer><p>&copy; 2026 asadokz18</p></footer>
</body>
</html>
```

Practice tasks

① Make a resume page ② Make a table timetable ③ Make a registration form ④ Make a 3-page website with links ⑤ Embed a video & a map.

21. Interview Questions & Answers

Q1. What is HTML?

HyperText Markup Language – the standard language for creating the structure of web pages using tags.

Q2. What is the difference between HTML and HTML5?

HTML5 adds semantic tags (header, nav, section), audio, video, canvas, SVG, new form inputs, and APIs (storage, geolocation) without plugins like Flash.

Q3. What is a tag and what is an element?

A tag is the markup `<p>`. An element = opening tag + content + closing tag.

Q4. What is an empty (void) tag? Give examples.

A tag without closing tag or content: `<br>`, `<hr>`, `<img>`, `<input>`, `<meta>`, `<link>`.

Q5. Difference between <b> and ?

`<b>` only makes text bold; `<strong>` also means **important** (helps screen readers & SEO). Same for `<i>` / `<em>`.

Q6. Difference between id and class?

id is unique – one element per page. class can be reused on many elements.

Q7. Block vs inline elements?

Block starts a new line and takes full width (div, p). Inline stays in line and takes only needed width (span, a).

Q8. What is the use of <!DOCTYPE html>?

Tells the browser to use HTML5 standards mode.

Q9. GET vs POST?

GET puts data in the URL (visible, limited, bookmarkable). POST sends data in the body (hidden, larger, secure for passwords).

Q10. What is semantic HTML?

Using tags that describe meaning (article, nav, footer) – better SEO & accessibility.

Q11. What is the alt attribute for?

Alternative text if image fails, for screen readers and SEO.

Q12. How do you open a link in a new tab?

`target="_blank"` (add `rel="noopener noreferrer"`).

Q13. localStorage vs sessionStorage vs cookies?

localStorage – permanent, ~5 MB. sessionStorage – until tab closes. Cookies – small (4 KB), sent to server with every request.

Q14. What is the viewport meta tag?

Controls page width / scale on mobile devices; needed for responsive design.

Q15. Canvas vs SVG? iframe? defer vs async?

Canvas = pixels by JS; SVG = vector shapes. iframe = page inside a page. defer = run after parsing, in order; async = run as soon as loaded.

22. Quick Cheat Sheet

Structure

<!DOCTYPE html> <html> <head> <title> <meta> <link> <style> <script> <body>

Text

<h1>–<h6> <p>
 <hr> <pre> <b> <i> <u> <mark> <small> <del> <ins> <sub> <sup> <q> <blockquote> <cite> <code> <kbd> <abbr>

Links & Media

<a href> <img src alt> <picture> <figure> <audio> <video> <source> <track> <iframe> <map> <area>

Lists

   <dl> <dt> <dd>

Tables

<table> <caption> <thead> <tbody> <tfoot> <tr> <th> <td> <colgroup> <col> – colspan, rowspan

Forms

<form> <label> <input> <textarea> <select> <option> <optgroup> <datalist> <button> <fieldset> <legend> <output> <progress> <meter>

Layout / Semantic

  <header> <nav> <main> <section> <article> <aside> <footer> <details> <summary> <dialog> <time>

Graphics & API

<canvas> <svg> localStorage · sessionStorage · geolocation · drag & drop · web workers

Most used attributes

href src alt id class style title lang name value type placeholder required disabled checked selected target rel width height controls autoplay loop colspan rowspan action method for data- aria-**

Learning roadmap

HTML → CSS → JavaScript → Responsive / Flexbox / Grid → Git & GitHub → Framework (React) → Projects!

Final advice

Type, break, fix, repeat. Build small projects every week. 💪 Practice is the secret of every great developer.

~ Thank you! Keep learning ~

by asadokz18 ✿